

Project 1 on Machine Learning: regression analysis, resampling methods and gradient descent

Tim Herrmann Holmen

FYS-STK4155

Department of Physics

Dette prosjektet er en vandring gjennom metodene innen dataanalyse som har ledet til det vi idag kaller maskinl ring. Hovedm let er   studere i detalj diverse regresjonsmetoder, som ordinary least squares, ridge regresjon og lasso regresjon. Resampling teknikker som bootstrap og kryssvalidering lar oss vurdere og velge den beste metoden for problemet. For   optimalisere parametrene (hvilke parametre er det her snakk om?: det er selve parametrene og hyperparametrene) vil vi bruke gradient descent, og teste ulike metoder for   utf re denne beregningen, som best r av derivasjon. Vi har studert newtons metode og automatiserte derivasjons metoder, for   optimalisere parametrene i modellene. Polynomene vi utvikler skal passe til en spesifikk en dimensjonal funksjon kalt Runges funksjon, med et st y element, dermed har vi det analytiske svaret tilgjengelig.

CONTENTS

I. INTRODUCTION

A. 2a The motivation.

The reason we want to fit a function to data is that we want to predict the value of the function to new unknown data points. For fitting a function to data points, we use polynomials, and we want to find the best polynomial that fits the data. The best polynomial is the one that minimizes the mean squared error (MSE) between the predicted values and the true values (for the known data). The MSE can be decomposed into three components: bias, variance and irreducible error.

$$MSE = Bias^2 + Var[\tilde{y}] + irreducible\ noise$$

The bias measures how far the average prediction is from the truth. The variance measures how much the prediction moves when the training set is redrawn. The irreducible error is the variance of the noise itself, which is baked into the data and the model cannot predict it. (, p. 62) The bias-variance tradeoff is the balance between bias and variance, in which we minimize the MSE. We do not want an overfitted model, which has low bias and high variance, nor do we want an underfitted model, which has high bias and low variance. Optimally, we want a model that is just right, with low bias and low variance. When fitting a polynomial to data, we want to find the right degree of the polynomial and the right parameters for each degree. For example if there is a linear relationship between the data points, and we can assume this by looking at the data, we use a second order polynomial to describe the data, and the parameters are the coefficients of the polynomial, which can be changed in accordance with the relationship between the data points. The parameters are found by minimizing the MSE between the

model prediction and datapoints, for avoiding overfitting, we can split the data into training and testing sets, to see how well the model performs on unseen data. We can use regularization methods, such as ridge regression and lasso regression, to regularize the model and avoid overfitting, and thus lower the variance, while maintaining a good bias. We can also use resampling methods, such as bootstrap and k-fold cross validation, to estimate the performance of the model on unseen data, and thus choose the best model for the data.

With lasso regression, we do not have the analytical solution for the gradient, as we do for OLS and Ridge, and we have to use gradient descent to find the optimal parameters. "Gradient descent is an optimization algorithm inspired by newtons method, which is used to find the minimum of a function. The function we want to minimize is the cost function, which is the MSE between the predicted values and the true values. The gradient descent algorithm uses the derivative of the cost function to find the direction of steepest descent, and then takes a step in that direction, with a step size determined by the learning rate. We can use different optimization algorithms, such as momentum, adagrad, rmsprop and adam, where the learning rate of the descent is dependent on the iteration number to improve the convergence of gradient descent." - AI After exploring the different methods, we can compare them with cross validation, and choose the best model for the data.

B. 2b What has been done and what i do.

The linear models with OLS and Ridge have been implemented, and both have closed form solutions, thus they can be solved without gradient descent. Lasso regression has been implemented, but it does not have a closed form solution, thus we have to use gradient descent to find the optimal parameters. The project starts with the imple-

mentation of OLS and Ridge regression, OLS uses the least square distance between model and data to find the optimal parameters, this is the cost function, and the optimal parameters are found by minimizing the cost function. Ridge regression does the same, but adds a regularization term to the cost function, which penalizes large parameters, and thus avoids overfitting. What ridge removes in variance, it adds in increased bias, which will be discussed as the bias-variance tradeoff. For exploring this theme we have employed resampling methods, the bootstrap and K-fold cross validation, to estimate the performance of the model on unseen data, and thus choose the best model for the data. The linear models OLS and Ridge are first treated separately, and then compared to each other with resampling methods to find the best degree of the polynomial, and the best regularization parameter, lambda, for Ridge. For lasso regression, we have to use gradient descent to find the optimal parameters, since it does not have a closed form solution. With this end in mind, we have implemented plain gradient descent, inspired by Newton's iteration method for solving differentials, and explored different optimization algorithms, such as momentum, adagrad, rmsprop and adam, where the learning rate of the descent is dependent on the iteration number. These methods are first implemented on OLS and Ridge, and compared to the closed form solutions of both, to see how well they perform, in terms of convergence (towards zero difference between closed form and gradient descent solutions), and data cost (number of iterations). In terms of hyperparameters, Ridge and Lasso both have a regularization term. The degree is a hyperparameter for all three models, and the learning rate is a hyperparameter for gradient descent. This project can be split in two parts, where first the Ridge and ordinary least square regression methods are used and evaluated. Then gradient descent is developed and used on all three methods, including Lasso, and the three methods are evaluated by k-fold cross validation. Underway the analytical solutions and sci-kit learn library are used and compared, to ensure numerical digit error, and in the last analysis only sci-kit learn library is used *RIGHT?*.

1. Linear Models

a. OLS

b. Ridge

c. Lasso

2. Resampling techniques

a. Bootstrap

b. Kfold cross validation

3. Optimization

a. Gradient descent

b. Momentum

c. Adagrad

d. RMSprop

e. Adam

f. Stochastic gradient descent

4. Cross-Validation

C. The structure of the report,

The last paragraph, one sentence per section, and where code can be found. Code can be found on the following Github repository: (?)

II. METHODS

A. The data

In this project, the input data is normally distributed, and the output data is generated by Runge's function, with added noise, Runge Function which is a one-dimensional function of second order.

$$f(x) = \frac{1}{1 + 25x^2}$$

From the beginning, we have created a relation between the input and output data. This is helpful since the project is research in machine learning methods, it is useful to know the exact solution. The noise is normally distributed, with a mean of 0 and a variable standard deviation. From the form of the Runge function, we can see that the function has a maximum at $x=0$, and the function is symmetric around this point, for values larger than $[-1,1]$, the function approaches zero. Thus we will focus on data which is normally distributed between -1 and 1.

The output is generated by adding noise σ to the runge function defined above, and the noise is normally distributed with a mean of 0 and a standard deviation of σ .

The input value x is transformed into a design matrix X , where the columns describe a polynomial of variable degree d , in such a way that the first column is x^0 , the intercept, the second column is x^1 , and so on, up to x^d . The rows of the matrix are the data points, and are thus copied and multiplied according to the polynomial degree, or the column of the matrix.

$$\mathbf{X} = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{pmatrix} \in \mathbb{R}^{n \times (d+1)}. \quad (1)$$

1. Splitting of the data

Train test split of skicit

2. Normalisation and scaling of the data

Unless explicitly told otherwise, such as in the bootstrap resampling method, the training data of all models are normalized.

B. Linear Regression

The basic idea of linear regression is, by using a polynomial $f(x)$ of fixed degree to describe the data points given the input data x , to adjust the coefficients in such a way that the difference between the predicted values of the model $f(x)$ and the true value $y(x)$. The coefficients are adjusted so that mean of the sum of the difference squared is minimised, which is the basis for the name least squares regression.

1. Ordinary Least Squares

Thus the cost function, for least squared, is written mathematically:

$$C(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 \quad (2)$$

(?, p. 81) This is the optimization problem, to adjust the parameter θ such that the cost function is minimized.

We see that our predicted model is characterized by $X\theta$, where θ contains the coefficients, the direct solution to this problem is:

$$\hat{\theta} = (X^T X)^{-1} X^T y \quad (3)$$

(?, p. 81) through singular value decomposition (?, p. 42), it can be shown that the best estimate for this θ , which minimizes the cost function, is the pseudoinverse of X multiplied by the output data y ,

$$\hat{\theta} = X^+ y \quad (4)$$

(?, p. 82) The pseudoinverse is solved by the numpy package `np.linalg.pinv(X)`, on the assumption that the columns of X are linearly independent, such that $X^T X = I$ and $X^+ = (X^T X)^{-1} X^T = I^{-1} X^T = X^T$.

It is safe to assume that the columns are linearly independent, since the number of columns (degrees d) is much smaller than the number of rows (data points n), the possibility of one column being a linear combination of the other columns is close to zero.

The data is split into training and test data as described above, and the training data is normalised as explained above. This ensures that the model can predict on the test data of X , and the predictions can be used against the test data of y to calculate the mean squared error MSE and coefficient of determination R^2 , how these values are calculated will be explained in measures of quality. This procedure is done for different degrees of the polynomial design matrix. The parameters are plotted against degree, and the predicted models for different degrees are plotted against the runge function, for different values of the noise σ .

2. Pseudocode

C. Ridge Regression

Ridge regression adds a penalty on the squared length of the parameter vector, and the cost function is;

$$C_{\text{Ridge}}(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2, \quad \lambda \geq 0. \quad (5)$$

with $\|\theta\|_2^2 = \sum_j \theta_j^2$. The closed form solution to this problem is, by dropping the factor $1/n$:

$$\hat{\theta}_{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T y \quad (6)$$

I is the $p \times p$ identity matrix. The penalty adds λ to the eigenvalues of $X^T X$. (?, p. 87) Supposing the design matrix is orthonormal, $X^T X = I$: then $\hat{\theta}_{OLS} = X^T y$ and

$$\hat{\theta}_{\text{Ridge}} = (I + \lambda I)^{-1} X^T y = \frac{1}{1 + \lambda} \hat{\theta}_{OLS} \quad (7)$$

So every coefficient is scaled by the same factor, and the estimator tends to zero as $\lambda \rightarrow \infty$. Ridge shrinks all coefficients proportionally; it never sets any of the exactly to zero.

1. Pseudocode Ridge

D. Lasso Regression

Replacing the 2-norm penalty of Ridge by a 1-norm gives the cost function

$$C(\theta)_{Lasso} = \frac{1}{n} \|y - X\theta\|_2^2 + \lambda \|\theta\|_1 \quad \|\theta\|_1 = \sum_{j=0}^{p-1} |\theta_j| \quad (8)$$

Which is the Lasso - least absolute shrinkage and selection operator. The Lasso sets coefficients exactly to zero, and thereby performs variable selection as part of the fitting. (Wahba, p. 90) The lasso has no closed form solution, since differentiating the cost function, requires the derivative of the absolute value of θ :

$$\frac{d|\theta|}{d\theta} = \text{sgn}(\theta) = \begin{cases} +1, & \theta > 0, \\ -1, & \theta < 0. \end{cases} \quad (9)$$

This equation is undefined at the origin, $\theta = 0$, thus the closed form solution is lost, but a solution can still be found through the soft thresholding operator and gradient descent methods, which will be explained, here is the soft thresholding operator for $X = I$

$$\hat{\theta}_i^{Lasso} = S_{\lambda/2}(y) = \text{sgn}(y) \max(|y| - \lambda/2, 0) \quad (10)$$

(Wahba, p. 91) The selection property makes the lasso the natural first tool when the number of parameters is large

and most features are believed irrelevant.

1. Pseudocode Lasso

E. Evaluation

1. Measures of quality

MSE is ..

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (11)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i. \quad (12)$$

F. Optimization

III. FINAL ANALYSIS

REFERENCES

- (Timholmen96), “Timholmen96/FYS-STK4155: Prosjektarbeid i faget FYS-STK4155: Anvendt dataanalyse og maskinl ring,” <https://github.com/Timholmen96/FYS-STK4155>.
M. H rth-Jensen (2016), *Machine Learning and the Physical Sciences* (Springer).